



# Локальные LLM

ОБЗОР · ВОЗМОЖНОСТИ · РАЗВЁРТЫВАНИЕ

## СТРУКТУРА ЗАНЯТИЯ

# Что разберём сегодня



### Обзор моделей

Qwen · Llama · Mistral · DeepSeek · Phi



### Локальные vs Облако

Бенчмарки · стоимость · приватность



### Размер vs Качество

7B / 14B / 32B / 70B · сравнительные таблицы



### Dense vs MoE

Сравнение архитектур и ограничений



### Квантизация

GGUF · GPTQ · AWQ · EXL2 · SpQR



### Развёртывание

llama.cpp · Ollama · vLLM · расчёт VRAM

# Почему локальные LLM в 2026?



## Конфиденциальность

Данные не выходят за пределы контура. Это особенно важно для корпоративных, медицинских и персональных данных.



## Экономия 30–150×

При объёме >100 млн токенов в месяц локальный инференс часто окупает стоимость оборудования.



## Offline-режим

Промышленные площадки, транспорт, закрытые контуры и полевые условия могут работать без доступа к интернету.

🔑 **2026: RTX 5090 = 32 ГБ GDDR7 · Apple M4 Max = 36 ГБ Unified Memory**

Впервые одна потребительская система позволяет комфортно запускать модели класса 30–32B с качеством, близким к сильным облачным моделям прошлого поколения.

# Ключевые открытые LLM: актуальная картина

| СЕМЕЙСТВО         | АВТОР      | РАЗМЕРЫ             | СИЛЬНАЯ СТОРОНА                                                 | ЛИЦЕНЗИЯ        |
|-------------------|------------|---------------------|-----------------------------------------------------------------|-----------------|
| Qwen3             | Alibaba    | 1.7B–235B           | Код, многоязычность (119 языков), развитые рассуждения          | Apache 2.0      |
| Llama 4           | Meta       | Scout /<br>Maverick | Очень длинный контекст у Scout и зрелая экосистема инструментов | Meta<br>License |
| Mistral / Mixtral | Mistral AI | 7B → 123B           | Высокая скорость у компактных моделей и сильная линейка MoE     | Apache 2.0      |
| DeepSeek R1/V3    | DeepSeek   | 7B → 671B           | Сильные результаты в математике и задачах рассуждения           | MIT             |
| Phi-4-mini        | Microsoft  | 3.8B                | Хорошая математика и компактность для 8 ГБ VRAM                 | MIT             |
| Gemma 4           | Google     | 4B · 12B · 27B      | Мультимодальность и практичное развёртывание 12B на 16 ГБ       | Gemma T.        |

# Локальные и облачные модели: где чей предел

| МОДЕЛЬ            | MMLU  | HUMANEVAL | MATH-500 | GPQA  | ТИП             |
|-------------------|-------|-----------|----------|-------|-----------------|
| GPT-4o            | ~88%  | ~90%      | ~76%     | ~50%  | Облако          |
| Claude Sonnet 4.6 | ~87%  | ~92%      | ~78%     | ~55%  | Облако          |
| Qwen3-235B-A22B ★ | ~87%  | ~90%      | ~93%     | 77.2% | Локальный       |
| DeepSeek R1       | ~84%  | ~87%      | 97.3%    | ~72%  | Локальный       |
| Llama 3.3 70B     | 86.0% | 88.4%     | ~77%     | ~46%  | Локальный       |
| Qwen3 32B         | ~83%  | ~87%      | ~88%     | ~60%  | Локальный       |
| Qwen2.5 7B        | 74.2% | 57.9%     | ~49%     | ~30%  | Локально · 5 ГБ |

**Облако:** оплата за токены и удобный быстрый старт, но стоимость заметно растёт при постоянной нагрузке

**Локально:** нет платы за токены, зато есть требования к VRAM, охлаждению и сопровождению

# Семейство Qwen2.5: как качество растёт вместе с размером

| МОДЕЛЬ       | MMLU  | HUMANEVAL | MATH  | VRAM (Q4) |
|--------------|-------|-----------|-------|-----------|
| Qwen2.5-1.5B | ~58%  | ~37%      | ~28%  | 1.5 ГБ    |
| Qwen2.5-3B   | ~65%  | ~55%      | ~35%  | 2.5 ГБ    |
| Qwen2.5-7B   | 74.2% | 57.9%     | 49.8% | 5 ГБ      |
| Qwen2.5-14B  | 79.7% | ~72%      | ~65%  | 9 ГБ      |
| Qwen2.5-32B  | ~83%  | 84.5%     | ~75%  | 20 ГБ     |
| Qwen2.5-72B  | 86.1% | ~88%      | ~77%  | 42 ГБ     |

💡 **Переход к Qwen3:** Qwen3-8B по многим сценариям сопоставима с Qwen2.5-14B — это почти выигрыш в один размерный класс без роста требований к памяти.



# Эволюция семейств Llama и Mistral

 LLAMA (META)

| МОДЕЛЬ           | ARCH      | MMLU  |
|------------------|-----------|-------|
| Llama 3.1 8B     | Dense     | ~73%  |
| Llama 3.1 70B    | Dense     | ~83%  |
| Llama 3.3 70B    | Dense     | 86.0% |
| Llama 4 Scout    | MoE 109B  | 79.6% |
| Llama 4 Maverick | MoE ~400B | ~82%  |

 MISTRAL AI

| МОДЕЛЬ                | ARCH        | MMLU |
|-----------------------|-------------|------|
| Mistral 7B            | Dense       | ~64% |
| Mixtral 8×7B          | MoE 46.7B   | ~70% |
| Mistral Small 3 (24B) | Dense       | ~80% |
| Mistral Large 3       | Dense ~123B | ~84% |

Mistral Small 3 и Large 3 распространяются по Apache 2.0 и подходят для коммерческого использования.

# Dense и Mixture of Experts (MoE)



## Dense (плотная)

Все параметры активны на каждом токене

VRAM = пропорционально числу параметров

Предсказуемое поведение и понятные требования к памяти

Примеры: Qwen2.5, Llama 3, Mistral 7B



## MoE (смесь экспертов)

На токен активны только  $k$  из  $N$  экспертов (обычно 2 из 8–64)

Вычислений на токен заметно меньше, чем у dense-моделей того же суммарного размера

VRAM = **все параметры**, потому что модель всё равно должна быть полностью загружена

Примеры: Mixtral, Llama 4, Qwen3.5, DeepSeek V3



**Ловушка MoE:** Llama 4 Maverick: суммарно около 400B параметров. Несмотря на то что на каждом токене активна лишь часть экспертов, в памяти приходится держать всю модель. Поэтому MoE снижает вычислительную нагрузку, но не решает проблему объёма VRAM.



# Методы квантизации: от простых к современным

## ТРАДИЦИОННЫЕ

**RTN** — округление до ближайшего значения. Метод очень быстрый, но на низкой битности может заметно ухудшать качество.

**INT8 / ONNX** — линейное масштабирование и классический формат для edge-устройств и инференса в продакшене.

**K-means Quant** — кластеризация весов с возможностью подбирать нестандартную битность и словарь значений.

## СОВРЕМЕННЫЕ (2023–2026)

**GPTQ** — посттренировочная квантизация с учётом чувствительности весов. Один из первых практически полезных методов для 3–4 бит.

**AWQ** — защита наиболее важных каналов. Обычно даёт более аккуратный компромисс между качеством и размером, чем базовый INT4.

**GGUF** — удобный формат для гибридного CPU/GPU-инференса. Q4\_K\_M обычно сохраняет около 92–93% качества FP16 при сильной экономии VRAM.

**EXL2 / SpQR / AQLM** — более тонкие схемы со смешанной битностью и меньшими потерями на 3–4 битах.

# Потеря качества при квантизации на примере 7B-модели



✓ Q5\_K\_M и выше — зона с минимальными потерями качества

⚖ Q4\_K\_M — лучший баланс между качеством и объёмом VRAM

⊖ Q2\_K/Q3\_K — компромиссный вариант только при жёстком дефиците памяти

# Как выбрать метод квантизации

| СЦЕНАРИЙ / ПЛАТФОРМА                 | МЕТОД          | ПРИМЕЧАНИЕ                                                                               |
|--------------------------------------|----------------|------------------------------------------------------------------------------------------|
| Ollama / llama.cpp / LM Studio       | GGUF<br>Q4_K_M | Универсальный вариант и лучшая стартовая точка                                           |
| Ollama, нужно больше качества        | GGUF Q6_K      | Требует больше памяти, но обычно даёт чуть более высокое качество                        |
| vLLM, NVIDIA, одиночный пользователь | AWQ 4-bit      | Один из лучших вариантов по соотношению качества и размера для GPU-сервера               |
| vLLM, multi-LoRA serving             | GPTQ-Int4      | Практичный вариант для сервинга с LoRA-адаптерами                                        |
| ExLlamaV2, максимум качества         | EXL2           | Смешанная битность по слоям ради максимального качества                                  |
| Apple Silicon / Mac                  | GGUF<br>(MLX)  | Удобно использовать через Ollama, LM Studio и экосистему Mac                             |
| NVIDIA Blackwell (RTX 5090+)         | NVFP4          | Новый формат для современных NVIDIA, ориентирован на максимальную пропускную способность |

💡 GGUF с imatrix-калибровкой от популярных авторов на Hugging Face часто дают небольшой, но полезный прирост качества при той же битности.

# Когда какой размер модели действительно нужен

## 1–4B

### Edge и встраиваемые устройства

Классификация текста  
NER, суммаризация  
IoT, мобильные устройства  
30–80 tok/s на GPU

Phi-4-mini · Qwen3-1.7B

## 7–14B

### Разработка, RAG и повседневные агенты

Программирование  
RAG-системы, перевод  
Агентные задачи  
15–40 tok/s на 16 ГБ

Llama 3.3 8B · Qwen2.5-Coder 14B

## 30–70B

### Максимум качества на локальном железе

Сложное рассуждение  
Длинный контекст >32K  
Юридический / мед. анализ  
5–20 tok/s на 32+ ГБ

Qwen3-32B · DeepSeek R1 32B

 Переход от 7B к 14B обычно даёт наиболее заметный прирост. Дальше качество растёт медленнее, а требования к VRAM увеличиваются значительно быстрее.



# Как быстро оценить потребность в VRAM

## Быстрая оценка:

$$\text{VRAM} \approx N_{\text{params}} \times \text{Bytes\_per\_param} \times 1.2 + \text{KV-cache}$$

## БАЙТ НА ПАРАМЕТР

| ТОЧНОСТЬ  | БАЙТ  | 7B МОДЕЛЬ |
|-----------|-------|-----------|
| FP16/BF16 | 2     | 14 ГБ     |
| INT8 / Q8 | 1     | 7 ГБ      |
| Q4_K_M    | ~0.56 | ~5 ГБ ✓   |
| Q2_K      | ~0.33 | ~3 ГБ     |

## KV-КЭШ — СКРЫТЫЙ ПОТРЕБИТЕЛЬ ПАМЯТИ

$$\text{KV} = 2 \times L \times H_{\text{kv}} \times d_{\text{head}} \times S \times B \times \text{bytes}$$

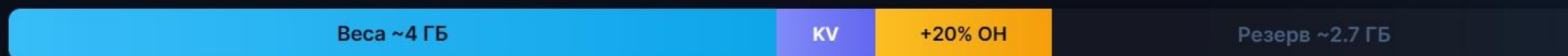
Llama 3.1 8B, 4K ctx: ~0.5 ГБ

Llama 3.1 8B, 128K ctx: ~16 ГБ

**70B, 32 польз-ля, 8K: ~83 ГБ (!)**

⚠ При длинном контексте объём KV-кэша может превысить объём самих весов модели.

Пример распределения памяти: 7B Q4\_K\_M, контекст 8K, видеокарта на 8 ГБ



# Лlama.cpp, Ollama и vLLM: что выбрать

| КРИТЕРИЙ                                     | LLAMA.CPP               | OLLAMA                 | VLLM                   |
|----------------------------------------------|-------------------------|------------------------|------------------------|
| Основа                                       | C++, собственный движок | Обёртка над llama.cpp  | Python, PagedAttention |
| Формат                                       | GGUF                    | GGUF (через llama.cpp) | Safetensors, GPTQ, AWQ |
| Установка                                    | Компиляция              | curl sh 3 мин          | pip install vllm       |
| Платформы                                    | CPU+GPU, Metal, ROCm    | CPU+GPU, Metal, ROCm   | NVIDIA (основной)      |
| Параллельные пользователи                    | 1 (очередь)             | 1–3                    | Десятки-сотни          |
| Пропускная способность при 10+ пользователях | Базовый уровень         | Базовый уровень        | 35× выше!              |
| LoRA-адаптеры                                | Есть                    | Ограниченно            | Полная поддержка       |
| Целевой сценарий                             | Edge, прототипирование  | Локальная разработка   | Production multi-user  |

 vLLM особенно силен под многопользовательской нагрузкой: PagedAttention и непрерывный батчинг дают резкий рост общей производительности.



# Практические рекомендации по объёму VRAM

## 8 ГБ VRAM

RTX 3070 · 4060 Ti · RX 7600 и близкие по классу карты

Llama 3.3 8B Q4\_K\_M ★

Qwen3-8B Q4\_K\_M

Phi-4-mini для задач, где важна математика

Mistral 7B, если приоритет — скорость отклика

```
ollama run llama3.3:8b
ollama run qwen3:8b
```

## 16 ГБ VRAM

RTX 3080 · 4070 Ti / 4080 · RX 6800 XT и близкие решения

Qwen3-14B Q5\_K\_M ★

Qwen2.5-Coder 14B Q5\_K\_M

Llama 3.3 8B Q8\_0

Gemma 4 12B для мультимодальных сценариев

```
ollama run qwen3:14b
ollama run qwen2.5-coder:14b
```

## 32 ГБ VRAM

RTX 5090 · A6000 Ada · конфигурации с несколькими GPU

Qwen3-32B Q5\_K\_M — один из лучших универсальных вариантов

Qwen2.5-Coder-32B Q4\_K\_M  
DeepSeek R1 32B для математики и сложных рассуждений

Mixtral 8×7B Q4\_K\_M

```
ollama run qwen3:32b
vllm serve Qwen/Qwen3-32B-AWQ
```

## Восемь практических принципов

1 Начиная с **Q4\_K\_M**: потери обычно умеренные, а экономия VRAM очень существенная. Для первого запуска это лучший компромисс.

2 Не выполняйте **повторную квантизацию**: ошибки округления накапливаются и ухудшают качество.

3 **KV-кэш** — скрытый потребитель памяти: при 128K+ контексте он может стать больше самих весов.

4 МоЕ не означает экономию VRAM: все параметры по-прежнему нужно **держат в памяти**.

5 Ищите **imatrix-калиброванные** GGUF-сборки: они часто оказываются точнее обычных при той же битности.

6 Для многопользовательского production-сценария ориентируйтесь на **vLLM** с PagedAttention; Ollama удобнее как локальный инструмент, а не как масштабируемый сервер.

7 **Гибридный подход часто выигрывает**: рутинные запросы обрабатываются локально, а самые сложные — в облаке.

8 **Apache 2.0 и MIT** — самые удобные лицензии для бизнеса; на них стоит обращать внимание при выборе модели для коммерческого применения.



# Запускайте локальные LLM осознанно

Локальные LLM стали практичным инструментом:  
оборудование доступнее, открытых моделей больше, а  
инструменты развёртывания заметно зрелее, чем ещё  
год назад.

 HuggingFace

 Ollama

 vLLM

 llama.cpp



